Python-Spickzettel – ICT703 Task 3 (Certiport IT Specialist: Python)

Prüfung: 30 Fragen · 50 Min (≈ 1:40 pro Frage) · Bestehen ab 70 % · Multiple Choice (4 Optionen, manchmal „Wähle 2“), Drop-down, Yes/No-Tabellen · open book, aber die Zeit ist knapp → schwere Fragen markieren, weiter, am Ende zurück.

Inhaltsverzeichnis

Domäne	Thema	Seite
–	Prüfungsinfos & Top-Fallen	1
D1	Datentypen, Casting, Arithmetik, Operator-Präzedenz, Logik, Vergleiche	2
D1	Slicing, String-/Listen-/Dict-Methoden, Datenstrukturen bauen, `is` / `in`	3
D2	if/elif/else, `range`, for/while, break/continue/pass, Tracing	4
D3	Dateien (`open`, Modi, `with`, os, csv), `input`/`print`, Formatierung, `sys.argv`	5
D4	Einrückung, Kommentare, Docstrings, pydoc, Funktionen	6
D5	Fehlerarten, Exceptions, try/except/else/finally, raise, unittest	7
D6	math, random, datetime, os/os.path, sys, io	8

Top-Fallen aus den Übungsfragen

1. **`is` ≠ `==`**: `is` = gleiches Objekt, `==` = gleicher Wert. `[2,4,7] is [2,4,7]` → False; `a = 45; b = 45; a is b` → True (kleine ints werden gecacht).
2. **`in` ist case-sensitive, sucht Teilstrings**: `'is' in 'This IS a test'` → **True** (steckt in „Th**is**“). `'h' in "Python"` → True.
3. **Truthiness**: `bool("")` False, `bool(" ")` True, `bool([False])` True, `bool(3)` True, `bool(0)` False.
4. **bool ist Zahl**: `True + 1` → 2, `3 + False` → 3, aber `type(False)` → `<class 'bool'>` (nicht int).
5. **`/` liefert immer float**, und sobald ein float mitrechnet, ist das Ergebnis float (7.0, nicht 7).
6. **Grenzen**: `range(1,5)` → 1–4 (ohne 5) · `randint(1,10)` → **mit** 10 · `randrange(1,10)` → ohne 10 · Slicing `[a:b]` → ohne b.
7. **`input()` liefert immer str** → vor dem Rechnen `int(...)`, sonst TypeError (`sum += "78"`).
8. `'6' + '5'` → `'65'` (str). `"Total: " + 5` → TypeError → `str(5)`.
9. **Default-Parameter nach normalen Parametern**: `def f(name, bound=2, letters):` → SyntaxError in der `def`-Zeile.
10. **Fehlender `:`** nach def/if/for/while → SyntaxError. **Nicht eingerückter Docstring/Block** → IndentationError. `items = [];` → Semikolon ist **erlaubt**.
11. **`break` beendet nur die innerste Schleife.** if/elif: **nur der erste** zutreffende Zweig läuft.
12. **In-place-Methoden geben `None` zurück**: `list.sort()`, `random.shuffle()`, `list.append()`. `sorted()` gibt eine neue Liste zurück.
13. **pydoc**: Modul-Docstring → DESCRIPTION, `__author__` → AUTHOR, `__copyright__` → **DATA**. `print(f.__doc__)` zeigt nur den Docstring der Funktion.
14. **Aufgaben mit „Which line causes an error?“**: Syntaxfehler stoppen alles schon vor dem Start; sonst die erste Zeile, die zur Laufzeit abstürzt.

D1 · Datentypen & Operatoren (1/2)

Datentypen & `type()`

Wert	`type()`	Hinweis	Wert	`type()`	Hinweis
`5`, `2**2`, `7 // 2`	`int`	ganze Zahl	`[1, 2]`	`list`	veränderbar, geordnet
`5.0`, `10 / 2`, `47.0`	`float`	`/` gibt immer float	`(1, 2)`, `(1,)`	`tuple`	unveränderbar; **`(1)` ist int!**
`"5"`, `'45'`	`str`	Zahl in Quotes = String	`{"a": 1}`, `{}`	`dict`	`{}` = leeres dict, kein set
`True`, `5 > 3`	`bool`	`True`/`False` großgeschrieben	`{1, 2}`, `set()`	`set`	keine Duplikate, ungeordnet
`2+5j`	`complex`	komplexe Zahl	`None`	`NoneType`	Funktion ohne `return`

Truthiness: False sind nur `0`, `0.0`, `""`, `[]`, `()`, `{}`, `set()`, `None`, `False`. Alles andere ist True (`" "`, `"0"`, `"False"`, `[False]`, `-1`).

Type Casting

Ausdruck	Ergebnis	Ausdruck	Ergebnis
`int("12")`	`12`	`int("3.5")` / `int("abc")`	ValueError
`int(3.9)` / `int(-3.9)`	`3` / `-3` (schneidet ab)	`float(5)`	`5.0`
`float("1.5")`	`1.5`	`str(3) + "4"`	`"34"`
`"5" + 5`	TypeError	`"ab" * 3`	`"ababab"`
`round(2.5)` / `round(3.5)`	`2` / `4` (zur geraden Zahl)	`round(3.14159, 2)`	`3.14`

Arithmetik

Operator	Bedeutung	Beispiel → Ergebnis
`/`	Division	`7 / 2` → `3.5` (immer float)
`//`	Floor Division (**abrunden**)	`7 // 2` → `3` · `-7 // 2` → `-4` · `7.0 // 2` → `3.0`
`%`	Modulo (Rest)	`17 % 5` → `2` · `-7 % 2` → `1` · `17 % True` → `0`
`**`	Potenz	`2 ** 3` → `8` · `2 ** 3 ** 2` → `512` (von rechts) · `-2 ** 2` → `-4`

Zuweisung: `=` weist zu, `==` vergleicht. `x += 1` (auch `-= *= /= //= %= **=`). `a, b = b, a` tauscht. Die letzte Zuweisung gewinnt.

Operator-Präzedenz (1 = zuerst)

1	2	3	4	5	6	7	8	9
`( )`	`**` (von rechts)	`-x` `+x` Vorzeichen	`* / // %` (links → rechts)	`+ -`	`== != < > <= >= is in` (verkettbar)	`not`	`and`	`or`

Beispiel: `6 // 4 % 5 + 2 ** 3 - 2.0` → `2**3 = 8` → `6//4 = 1` → `1 % 5 = 1` → `1 + 8 - 2.0` = **`7.0`**. Und `10 + 5 % True` → `10`, `True + 5 ** 2` → `26`.

Logik: `and` · `or` · `not`

- `and`: beide True · `or`: mindestens einer True · `not`: dreht um. `True or False and False` → True (`and` bindet stärker).
- Kurzschluss: gibt einen **Operanden** zurück: `5 and True` → `True` · `0 and x` → `0` · `0 or "x"` → `"x"` · `5 or x` → `5`.

Vergleich: `==` `!=` `<` `>` `<=` `>=` → bool

`1 == 1.0` → True · `"1" == 1` → False · Strings werden Zeichen für Zeichen verglichen: `"apple" < "banana"` → True · `"Z" < "a"` → True (Großbuchstaben kommen zuerst) · `"10" < "9"` → True (String, keine Zahl!) · `"abc" < "abcd"` → True.

D1 · Datentypen & Operatoren (2/2)

Indexing & Slicing: `seq[start:stop:step]` (stop exklusiv)

```
s = "Python"     P   y   t   h   o   n
Index:           0   1   2   3   4   5
negativ:        -6  -5  -4  -3  -2  -1
```

`s[0]` → `'P'`	**`s[-1]` → `'n'`**	**`s[1:4]` → `'yth'`**	**`s[:2]` → `'Py'`**
`s[-3:]` → `'hon'`	`s[::2]` → `'Pto'`	`s[::-1]` → `'nohtyP'`	`s[2:100]` → `'thon'`

`s[10]` → IndexError, beim Slicing gibt es dagegen nie einen Fehler. Strings sind unveränderbar: `s[0] = "X"` → TypeError.
Beispiel: `w = "The cow has spots and walks slowly."` → `w[-7:]` = `'slowly.'`, `w[:8]` = `'The cow '`, `w[4:7]` = `'cow'` → zusammen `'slowly.The cow cow'`.

String-Methoden (geben neue Strings zurück)

Methode	Beispiel → Ergebnis	Methode	Beispiel → Ergebnis
`upper()`, `lower()`	`"Ab".upper()` → `'AB'`	`title()` / `capitalize()`	`'Hello World'` / `'Hello world'`
`find(x)` / `rfind(x)`	erste / letzte Position, **-1**, wenn nicht da	`index(x)`	wie `find`, aber **ValueError**
`count(x)`	`"banana".count("a")` → 3	`replace(a, b)`	`"banana".replace("a","o")` → `'bonono'`
`split(sep)`	`"a,b".split(",")` → `['a', 'b']`	`sep.join(liste)`	`"-".join(["a","b"])` → `'a-b'` (nur str!)
`strip()`, `lstrip()`, `rstrip()`	Leerzeichen am Rand weg	`startswith()`, `endswith()`	→ bool
`isalpha()`, `isdigit()`	`"demo".isalpha()` → True	`isalnum()`, `isupper()`, `islower()`	→ bool

Beispiel: `s = 'To Python or not to Python, that is the question!'` → `len(s)` = 49, `s.find('on')` = 7, **`s.rfind('on')` = 46**, `s.find('question')` = 40.

Listen (mutable)

Methode	Wirkung	Methode	Wirkung
`append(x)`	1 Element hinten anhängen	`extend(liste)`	mehrere Elemente anhängen
`remove(x)`	erstes x löschen (ValueError, wenn fehlt)	`pop()` / `pop(i)`	letztes / an i entfernen **und zurückgeben**
`sort()` / `sort(reverse=True)`	in-place, gibt **None** zurück	`sorted(l)`	neue sortierte Liste
`del l[i]`, `clear()`	per Index löschen / leeren	`index(x)`, `count(x)`, `reverse()`	Position, Anzahl, umdrehen
`len()`, `min()`, `max()`	Länge, Min, Max	`sum()`	nur mit Zahlen

`insert(i, x)` fügt an Index i ein: `['apple','banana','cherry'].insert(1, 'orange')` → `['apple', 'orange', 'banana', 'cherry']`. `[1] + [2]` → `[1, 2]` · `[0] * 3` → `[0, 0, 0]` · `b = a` → dasselbe Objekt (`a is b` True) · `b = a.copy()` oder `a[:]` → neues Objekt. **Datenstrukturen bauen:** leer `lst = []` + `append` in einer Schleife · List Comprehension: `[x * 2 for x in range(5) if x % 2 == 0]` → `[0, 4, 8]` · verschachtelt: `m = [[1, 2], [3, 4]]` → `m[1][0]` → 3.

Dict · Tuple · Set

- **dict**: `d["k"]` (KeyError, wenn der Key fehlt) · `d.get("k")` → None · `d.get("k", 0)` → 0 · `d.keys()`, `d.values()`, `d.items()` · `d["neu"] = 1` · `del d["k"]` · `for k, v in d.items():`
- **`in` prüft bei dicts nur die Keys**: `2 in {2: "C++"}` → True, `"C++" in {2: "C++"}` → False.
- **tuple**: unveränderbar (`t[0] = 1` → TypeError) · Unpacking: `a, b = (1, 2)`.
- **set**: `{1,2} | {2,3}` Vereinigung · `&` Schnittmenge · `-` Differenz · `s.add(x)`.

Identität: `a = 45; b = 45; a is not b` → False · `c = [2,4,7]; d = [2,4,7]; c is d` → False, `c == d` → True · `"test" is "test".upper()` → False · `x is None` ist der übliche Einsatz von `is`.

D2 · Ablaufsteuerung: Entscheidungen & Schleifen

if / elif / else

```
if loan >= 1000:            # Doppelpunkt + Einrückung!
    rate = 0.4
elif loan > 550:            # nur geprüft, wenn alles davor False war
    rate = 0.2
else:                       # wenn nichts zutrifft
    rate = 0
```

- Nur **der erste** zutreffende Zweig läuft, danach ist der ganze Block fertig → **Reihenfolge entscheidet**. Bereiche mit `>=` von groß nach klein ordnen, mit `<=` von klein nach groß.
- Grenzen genau lesen: `> 550` → 550 nein, **551 ja** · `< 750` → 750 nein · `<` vs `<=`!
- Verkettung: `if 1000 <= x <= 2000:` · Compound: `if sales > 100000 and new_customer:` · Einzeiler: `msg = "ja" if x > 0 else "nein"`
- **Nested**: Die Einrückung entscheidet, zu welchem `if` ein `else` gehört. Groß/klein ignorieren: `if acc.lower() == 'a':` → `'A'` passt auch.

range(start, stop, step)

Aufruf	Werte	Aufruf	Werte
`range(5)`	0, 1, 2, 3, 4	`range(2, 10, 3)`	2, 5, 8
`range(1, 5)`	1, 2, 3, 4 (4 Durchläufe)	`range(10, 0, -3)`	10, 7, 4, 1
`range(1, 13)`	1 … 12 (12 Monate)	`range(len(liste))`	alle Indizes

for & while

```
for word in w_list:              # über Elemente
for i, v in enumerate(lst):      # Index + Wert
for k, v in d.items():           # dict: Key + Value
while guess != "Riga" and count < 3:   # läuft, solange True (auch mit and/or/not)
    guess = input("Guess again ")
    count += 1                   # Zähler IM Block, sonst Endlosschleife!
```

break · continue · pass

Keyword	Wirkung
`break`	verlässt die **innerste** Schleife sofort
`continue`	überspringt den Rest des aktuellen Durchlaufs → nächster Durchlauf
`pass`	macht nichts, Platzhalter (ein leerer Block wäre ein IndentationError)
`for … else` / `while … else`	`else` läuft nur, wenn **kein** `break` passiert ist

Schleifen tracen: Variablen in einer Tabelle mitschreiben

```
w_list = ['dog', 'bear']; x = 0
for word in w_list:
    for i in range(len(word)):
        x = x + 1
        if x > 6:
            break               # beendet nur die innere Schleife
```

dog: x = 1, 2, 3 · bear: x = 4, 5, 6, 7 → `break` → die äußere Schleife ist ohnehin fertig → **x = 7**.

Summe ohne bestimmte Elemente (Drop-down-Frage):

```
for e in elements:
    if e in items:       # items = dict → prüft die Keys 1–6
        continue         # überspringen (pass würde NICHT überspringen)
    count += e
```

Einmaleins 4 × 1 bis 4 × 10:

```
while x < 11:                                          # x = 1 … 10
    print(str(n) + ' x ' + str(x) + ' = ' + str(n*x))
    x += 1                                             # eingerückt, sonst Endlosschleife
```

Fallen dabei: `+ (n*x)` ohne `str()` → TypeError · `while x <= 11` → ein Durchlauf zu viel (4 x 11).

D3 · Ein- & Ausgabe

`open(datei, modus)`

Modus	Bedeutung	Datei fehlt	Datei existiert
`"r"` (Default)	lesen	FileNotFoundError	liest ab dem Anfang
`"w"`	schreiben	wird erstellt	**überschreibt** (alter Inhalt ist weg)
`"a"`	anhängen	wird erstellt	schreibt ans Ende
`"x"`	exklusiv erstellen	wird erstellt	FileExistsError

Zusätze: `+` = lesen **und** schreiben (`"r+"`, `"w+"`, `"a+"`) · `b` = binär/bytes (`"rb"`, `"wb"`) · `t` = Text (Default).

Lesen & Schreiben

Befehl	Wirkung	Befehl	Wirkung
`f.read()` / `f.read(5)`	alles als ein String / 5 Zeichen	`f.readlines()`	Liste aller Zeilen
`f.readline()`	**eine** Zeile (mit `\n`)	`for line in f:`	Zeile für Zeile
`f.write(s)`	**nur str**, kein auto-`\n`	`f.writelines(liste)`	Liste von str, kein `\n`
`f.close()`	danach lesen → ValueError	`print(x, file=f)`	print in Datei

```
with open("log.txt", "a") as f:     # with schließt automatisch, auch bei Fehlern
    f.write(text + "\n")
import os
if os.path.isfile("log.txt"):      # oder os.path.exists()
    f = open("log.txt", "a")       # existiert → anhängen
else:
    f = open("log.txt", "w")       # neu anlegen
if os.path.exists("old.txt"):
    os.remove("old.txt")           # Datei löschen (os.rmdir = leeren Ordner löschen)
```

CSV (`import csv`)

```
with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Mark", "A"])      # eine Zeile
    writer.writerows(students)          # Liste von Listen → mehrere Zeilen
with open("students.csv", newline="") as f:
    for row in csv.reader(f):           # row ist eine Liste
        print(row)
```

Falsch: `writer.write()` (gibt es nicht) · Modus `"wb"` (csv braucht Text) · `f.writerows()` (Datei-Objekt hat das nicht).

Konsole: `input()` & `print()`

- `name = input("Name: ")` → **immer str** → `int(input())`, `float(input())`.
- `print(a, b)` → Leerzeichen dazwischen · `print(a, b, sep="-", end="")` (`end` Default `"\n"`) · `print("Total: " + 5)` → TypeError.
- **Kommandozeile** (`import sys`): `python script.py Anna 5` → `sys.argv == ['script.py', 'Anna', '5']` · `sys.argv[0]` = Skriptname · alles **str**.

Formatierung

Code	Ergebnis	Code	Ergebnis
`f"{name} hat {n}"`	f-String	`"{} {}".format(a, b)`	nach Position
`f"{3.14159:.2f}"`	`'3.14'`	`"{1} {0}".format("a", "b")`	`'b a'`
`f"{42:5d}"`	`' 42'`	`"{n}".format(n="X")`	`'X'`
`f"{7:03d}"`	`'007'`	`"%s ist %d" % ("a", 5)`	`'a ist 5'`
`f"{'ab':>5}"` / `<` / `^`	rechts / links / zentriert		
`f"{1234567:,}"` / `f"{0.25:.1%}"`	`'1,234,567'` / `'25.0%'`	`"%.2f" % 3.14159`	`'3.14'`

D4 · Code-Dokumentation & Struktur

Einrückung & Leerraum

- Einrückung (Standard: 4 Leerzeichen) definiert Blöcke, so wie `{ }` in anderen Sprachen. Uneinheitliche Einrückung → **IndentationError**.
- Nach `if`, `elif`, `else`, `for`, `while`, `def`, `class`, `try`, `except`, `with` kommt `:` und dann ein eingerückter Block.
- Leerzeilen (white space) dienen nur der Lesbarkeit. Lange Zeilen umbrechen mit `\` am Zeilenende oder innerhalb von `( )`.

Kommentare & Docstrings

```
"""
Modul-Docstring = erste Anweisung der Datei  → pydoc: NAME (1. Zeile) + DESCRIPTION
"""
__author__ = "Jim tester"                  # → AUTHOR
__copyright__ = "Copyright 2019"           # → DATA  (!)
__version__ = "1.0"                        # → VERSION

# Einzeiliger Kommentar, wird ignoriert (Code „auskommentieren“)
def get_random():
    """returns a random number"""          # Funktions-Docstring: 1. Zeile im Body, EINGERÜCKT
    return random.randint(1, 100)
```

- Docstring nach `def` nicht eingerückt → `IndentationError: expected an indented block`.
- `print(get_random.__doc__)` → **nur** der Docstring dieser Funktion (ohne Modul-Docstring, ohne Signatur).
- `help(get_random)` → Signatur + Docstring.

pydoc

Befehl	Wirkung	Befehl	Wirkung
`python -m pydoc Example`	Doku im Terminal	`import Example` → `help(Example)`	Doku im Interpreter (erst importieren!)
`python -m pydoc -w Example`	HTML-Datei schreiben	`python -m pydoc -p 1234`	Doku-Webserver auf Port 1234
`python -m pydoc -k suchwort`	Module nach Stichwort suchen	`python -m pydoc pass`	Hilfe zu einem Keyword

Sektionen in dieser Reihenfolge: `NAME` (Modulname – erste Docstring-Zeile) · `DESCRIPTION` (Modul-Docstring) · `FUNCTIONS` (Signatur + Docstring jeder Funktion; Funktionen ohne Docstring nur mit Signatur) · `DATA` (globale Variablen inkl. `__copyright__`) · `VERSION` · `AUTHOR` · `CREDITS` · `FILE`.

Funktionen

```
def subtotal(amount, tax=.08):          # tax hat einen Default-Wert
    """Berechnet die Summe inkl. Steuer."""
    return amount * (1 + tax)           # ohne return → None

subtotal(300)                     # tax = .08
subtotal(300, .06)                # positional
subtotal(tax=.06, amount=300)     # Keyword-Argumente, Reihenfolge egal
```

Regel / Code	Folge	Regel / Code	Folge
Default-Parameter **nach** normalen: `def f(a, b=2, c):`	SyntaxError	Aufruf: positional vor keyword: `f(a=1, 2)`	SyntaxError
Falsche Anzahl an Argumenten	TypeError	Aufruf vor der Definition	NameError
`def f(*args)`	beliebig viele → **Tupel**	`def f(**kwargs)`	beliebig viele keyword → **dict**
`return a, b`	gibt Tupel `(a, b)` zurück	Code nach `return`	wird nie ausgeführt
`def todo(): pass # später`	leere Funktion (Platzhalter)	Variablen in Funktionen sind lokal	ändern global: `global x`

D5 · Fehlerbehebung & Ausnahmebehandlung

Fehlerarten

Art	Wann?	Beispiele
Syntax error	vor dem Start, nichts läuft	fehlender `:`, Klammer/Quote fehlt, `=` statt `==` im if
Runtime error	beim Ausführen → Absturz	Division durch 0, Datei fehlt, str + int
Logic error	läuft, Ergebnis falsch	`<` statt `<=`, elif-Reihenfolge, `"riga" != "Riga"` → `.upper()`

Wichtige Exceptions

Exception	Auslöser	Exception	Auslöser
`SyntaxError`	`def f(a, b)` ohne `:` · `def f(a=1, b):`	`IndentationError`	Einrückung fehlt/falsch
`TypeError`	`"5" + 5`, `len(5)`, Argumentanzahl	`ValueError`	`int("abc")`, `lst.remove(99)`
`NameError`	Name nicht definiert / Tippfehler	`ZeroDivisionError`	`x / 0`, `x // 0`, `x % 0`
`IndexError`	`lst[10]` (zu groß)	`KeyError`	`d["fehlt"]`
`FileNotFoundError`	`open("fehlt.txt")` mit `"r"`	`AttributeError`	`x = 5; x.append(1)`

Beispiel: `items.append(input())` speichert Strings → in `sum += i` wird int + str gerechnet → **TypeError in der Zeile mit `sum += i`**.

try / except / else / finally

```
try:
    result = a / b                    # riskanter Code
except ZeroDivisionError:             # spezifische Exceptions zuerst!
    print("You cannot divide by zero")
except (ValueError, TypeError) as e:  # mehrere auf einmal, e = Fehlermeldung
    print("Fehler:", e)               # except Exception: fängt alles → immer zuletzt
else:
    print("Nur wenn KEIN Fehler")     # muss direkt nach den excepts kommen
finally:
    print("Läuft IMMER")              # auch bei Fehler oder return
```

Ablauf: Ohne Fehler: ganzer `try` → `else` → `finally`. Mit Fehler: `try` bis zur Fehlerzeile → **erster** passender `except` → `finally`. Reihenfolge immer try → except → else → finally. **raise:** `if b == 0: raise ZeroDivisionError("This did not work")` · `raise ValueError("Ungültig")` · `raise` allein im except-Block wirft die Exception erneut. Danach läuft kein Code mehr, außer die Exception wird gefangen.

unittest

```
import unittest
class TestCalc(unittest.TestCase):        # erbt von TestCase; setUp/tearDown: vor/nach jedem Test
    def test_add(self):                   # Name muss mit test beginnen!
        self.assertEqual(add(2, 3), 5)            # Funktion testen
        self.assertEqual(Konto(100).abheben(30), 70)   # Methode einer Klasse genauso
if __name__ == '__main__':
    unittest.main()
```

Assert	prüft	Assert	prüft
`assertEqual(a, b)` / `assertNotEqual`	`a == b`	`assertTrue(x)` / `assertFalse(x)`	`bool(x)`
`assertIs(a, b)` / `assertIsNone(x)`	`a is b` / `x is None`	`assertIn(a, b)` / `assertIsInstance(a, T)`	`a in b` / Typ
`assertRaises(Error, func, arg)`	löst Fehler aus	`self.fail("msg")`	Test schlägt sofort fehl

Ergebnis-Zeichen: `.` ok · `F` Failure (Assert falsch) · `E` Error (Exception) · `s` skipped. **Skippen:** `@unittest.skip("Grund")` immer · `@unittest.skipIf(Bedingung, "msg")` → skip, wenn **True** · `@unittest.skipUnless(Bedingung, "msg")` → skip, wenn **False** · `self.skipTest("msg")` im Test. Steht `self.fail()` **vor** `skipTest()` → Failure, kein Skip! **Beispiel:** `skipUnless(2 > 5)` → skip · `if "demo".isalpha(): self.skipTest()` → skip · `skipIf("demo".upper() != "DEMO")` → läuft (und schlägt fehl).

D6 · Module & Tools

Import: `import math` → `math.sqrt(9)` · `from math import sqrt` → `sqrt(9)` · `import math as m` → `m.pi` · ohne Import → NameError.

math

Funktion	Ergebnis	Funktion	Ergebnis
`math.ceil(3.2)`	`4` (aufrunden)	`math.sqrt(9)`	`3.0` (float)
`math.floor(3.8)` / `floor(-12.5)`	`3` / `-13` (abrunden)	`math.isqrt(10)`	`3` (int-Wurzel)
`math.trunc(-12.5)`	`-12` (abschneiden Richtung 0)	`math.pow(2, 3)`	`8.0` (immer float; `2**3` → `8`)
`math.fabs(-12)`	`12.0` (Betrag als float)	`math.pi`, `math.e`	Konstanten, **ohne** `()`
`math.fmod(-13, 3)`	`-1.0` (`-13 % 3` → `2`!)	`math.isnan(x)`, `math.nan`	Not a Number
`math.frexp(3)`	`(0.75, 2)` → $0.75 \cdot 2^2$	`math.factorial(5)`	`120`

Kreisfläche: `math.pi * r ** 2` · `print(math.ceil(3.2), math.floor(3.8))` → **`4 3`**

random

Funktion	Ergebnis	Funktion	Ergebnis
`random.random()`	float 0.0 ≤ x < 1.0	`random.uniform(1, 5)`	float zwischen 1 und 5
`random.randint(1, 10)`	int 1 … 10 (**inklusive** 10)	`random.randrange(10)` / `(1, 10, 2)`	0 … 9 / 1, 3, 5, 7, 9 (stop exklusiv)
`random.choice(liste)`	**ein** zufälliges Element	`random.sample(liste, 3)`	3 **verschiedene** Elemente
`random.choices(liste, k=3)`	3 Elemente **mit** Wiederholung	`random.shuffle(liste)`	mischt in-place → `None`
`random.seed(42)`	reproduzierbar	`random.SystemRandom()`	kryptografisch sicher

Passwort aus 3 Wörtern: `"".join(random.SystemRandom().choice(my_words) for i in range(3))` Falle: `lst[random.randint(0, len(lst))]` → IndexError möglich → `randint(0, len(lst)-1)` oder `randrange(len(lst))`.

datetime

```
from datetime import datetime, date, timedelta   # oder import datetime → datetime.datetime.now()
now = datetime.now()                     # 2026-10-14 09:05:03.782696
now.strftime("%H:%M:%S")                 # Objekt → String   (f = format)
datetime.strptime("14/10/2026", "%d/%m/%Y")   # String → Objekt   (p = parse)
now.weekday()        # Mo = 0 … So = 6      now.isoweekday()   # Mo = 1 … So = 7
now.year; now.month; now.day; now.hour; date.today(); now + timedelta(days=7)
(date(2026, 12, 24) - date(2026, 10, 14)).days   # Differenz = timedelta → .days → 71 (int)
```

`%Y` 2026	`%y` 26	`%m` 10 (Monat)	`%d` 14	`%H` 09 (24 h)	`%I` 09 (12 h)	`%M` 05 (Minute)	`%S` 03
`%A` Wednesday	`%a` Wed	`%B` October	`%b` Oct	`%p` AM/PM	`%j` Tag im Jahr		

os · os.path · sys · io

Befehl	Wirkung	Befehl	Wirkung
`os.getcwd()` / `os.chdir(p)`	aktueller Ordner / wechseln	`os.path.exists(p)`	Datei **oder** Ordner da
`os.listdir(p)`	Inhalt als Liste	`os.path.isfile(p)` / `isdir(p)`	ist Datei / ist Ordner
`os.mkdir(p)` / `makedirs(p)`	Ordner / mit Unterordnern	`os.path.join("a", "b.txt")`	`'a/b.txt'`
`os.remove(p)` / `os.rmdir(p)`	Datei / leeren Ordner löschen	`os.path.basename(p)` / `dirname(p)`	Dateiname / Ordnerteil
`os.rename(a, b)`	umbenennen	`os.path.getsize(p)` / `splitext(p)`	Bytes / `('name', '.txt')`
`sys.argv` / `sys.exit()`	Argumente (str) / beenden	`sys.version` / `platform` / `path`	Version / OS / Modulpfade
`sys.stdout.write(s)`	Ausgabe ohne `print`	`io` / `io.StringIO()`	`open()` ohne Import / Text im RAM